

Implementation of an IoT-based Battery Management System (BMS) Using Dual-Core ESP32

Abstract— This paper implemented various technologies that companies want at the undergraduate level. Rather than focusing on a single technology, this work incorporates multiple technologies into a cohesive project, providing a broad overview of capabilities that align with industry needs. If you proceed with a project based on this paper, you will have no difficulty in reviewing the technologies you have learned and becoming the talent that companies need. The system utilizes a dual-core ESP32 microcontroller, which continuously monitors battery current through an INA219 sensor. If the current exceeds a predefined safety threshold, a relay is triggered to disconnect the circuit, thereby preventing potential overheating or fire. Wireless communication is enabled through the MQTT protocol, providing real-time data transfer to a Django-based web interface for monitoring. FreeRTOS is employed to manage multitasking efficiently, allowing sensor data to be processed and transmitted every 100ms without overloading the system. The system shows the use of low-power MQTT technology, real-time data transmission, and remote monitoring to enhance battery safety and reliability.

Keywords—ESP32, FreeRTOS, MQTT, IOT

I. INTRODUCTION

Battery safety is critical in modern applications such as electric vehicles, energy storage systems, and portable electronics. One of the most significant risks is an overcurrent situation that can lead to overheating or even fires if left unchecked. Battery Management Systems (BMS) play an essential role in mitigating these risks by continuously monitoring battery conditions, including voltage, current, and temperature[1]. Overcurrent protection is a crucial feature of BMS, where the system must immediately isolate the battery from the circuit when current levels exceed safe thresholds. This project focuses on implementing a battery fire prevention system within a BMS, where the primary function is to monitor the battery's current and disconnect the circuit via a relay if an overcurrent condition is detected. The system is built using a dual-core ESP32 microcontroller, which provides the necessary computational power to handle real-time monitoring and wireless communication simultaneously. The ESP32's Wi-Fi capability, combined with the lightweight MQTT protocol, enables efficient wireless data transmission to a remote server[2]. This server is equipped with a Django-based web interface for real-time monitoring and control. The use of FreeRTOS ensures that all tasks—sensor data collection, relay control, and wireless communication—are handled in parallel without performance bottlenecks. The source code and more detail result for this project is available on GitHub[7].

II. RESEARCH BACKGROUND

In high-performance battery systems, safety measures must be implemented to protect against overcurrent situations, which can damage the battery or lead to fire hazards. The rapid detection and mitigation of these risks are essential for preventing accidents. Traditional BMS solutions may rely on wired communication for monitoring, but wireless communication offers more flexibility and remote access, especially in IoT applications.

MQTT (Message Queuing Telemetry Transport) has become a popular choice for IoT projects due to its lightweight nature and low power consumption, making it ideal for applications that require frequent, real-time data transmission[3][4]. This project utilizes MQTT to send battery data, including current levels and relay status, to a remote server where the system can be monitored in real time. By combining this with FreeRTOS, which enables concurrent task handling, the system effectively monitors current levels every 100ms and immediately triggers the relay to cut off the circuit if the current exceeds the predefined threshold.

Using dual-core architecture is effective for processing large amounts of data in real-time embedded systems[5]. One core handles sensor data processing, while the other manages MQTT communication. This separation of tasks ensures that the system remains responsive, even under heavy data loads, and can react to overcurrent conditions in real time.

III. SYSTEM DESIGN

A. Hardware Configuration

ESP32 Microcontroller: The dual-core structure enables FreeRTOS multitasking, with Core 0 dedicated to real-time sensor data collection and relay control, while Core 1 manages wireless communication through MQTT.

INA219 Sensor: This sensor measures the current and voltage flowing through the battery. It uses I2C to communicate with the ESP32 and provides real-time data on battery electrical performance. The data is processed every 100ms to detect any overcurrent condition.

Relay: The relay is responsible for disconnecting the battery from the circuit in the event of an overcurrent condition. It is triggered when the system detects that the battery current exceeds the safety threshold (configured at 400mA). The relay keeps that the battery is safely isolated, preventing further damage.

Django Web Server: The server shows a real-time monitoring interface that displays key metrics such as current, voltage, and relay state, along with alerts if current levels approach unsafe thresholds.

B. Software Structure

The software structure is based on FreeRTOS, which allows for efficient multitasking and management of multiple tasks across the ESP32's dual cores. The system monitors current levels in real time and triggers the relay when necessary. Additionally, the system communicates wirelessly via MQTT to transmit data to a remote server for real-time monitoring.

TaskSensorRead: This task collects data from the INA219 sensor every 100ms, including the current and voltage values. The collected data is checked against the safety threshold, and if the current exceeds the limit, the relay is triggered to disconnect the circuit. This task runs on Core 0, ensuring real-time sensor data processing.

TaskMQTT: This task manages the transmission of sensor data and system status (such as relay activation) to the remote server using the MQTT protocol. Data is transmitted wirelessly every 100ms, and the server displays real-time data via a web interface. Core 1 is work to handling this task, ensuring smooth communication without interfering with sensor data processing.

TaskCAN: Manages CAN protocol-based communication, periodically sending messages containing relay and system status updates.

Django Web Server: The Django server provides a web interface for real-time monitoring of battery conditions. The system displays key metrics such as current, voltage, and the status of the relay. It also generates alerts if the current exceeds the threshold and allows the user to monitor the system from any internet-connected device.

IV. CURRENT MONITORING AND PROTECTION MECHANISM

A. Current Threshold Monitoring

The INA219 sensor continuously monitors the current in the battery circuit. The system checks the current value every 100ms to ensure that it remains within safe limits. In this implementation, The maximum measurable current of INA219 is 320mA, but since the error range is not large, the threshold value was set to 400mA. (When measuring 400mA, the error range is approximately $\pm 3\text{mA}$.)[6], defined as the maximum allowable current before triggering the relay.

B. Relay Activation for Circuit Disconnection

The relay is activated whenever the current exceeds the 400mA threshold. Upon detection of an overcurrent condition, the system triggers the relay, cutting off the circuit to prevent further current from flowing into the battery. This action occurs within milliseconds of detection.

The system also monitors the current after the relay is activated. Once the current returns to safe levels, the relay can either be manually reset via the web interface or automatically, depending on the system configuration. This makes that the battery is not exposed to further damage during hazardous conditions.

V. WIRELESS COMMUNICATION USING MQTT

A. MQTT for Low-Power Wireless Communication

MQTT was selected for its low-power consumption and lightweight nature, making it ideal for real-time IoT applications like BMS. The ESP32 connects to the MQTT broker via Wi-Fi and continuously publishes data every 100ms. The data includes the current and voltage readings from the INA219 sensor and the status of the relay (whether the circuit has been disconnected).

B. Real-Time Monitoring and Control via Web Interface

The Django-based web server subscribes to MQTT topics to receive both raw and averaged sensor data for real-time monitoring of battery status. The interface show current and voltage metrics along with relay status, updating in real time to reflect the system's operation. Should current levels approach unsafe limits, an alert is displayed, allowing users to remotely reset the relay after a safety-triggered shutdown or to perform an emergency stop if necessary. This web interface gives continuous oversight and control, facilitating battery management from any internet-connected device.

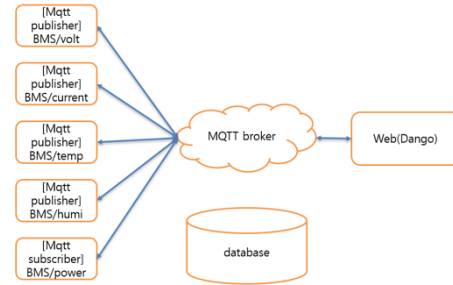


Figure 1. wireless communication architecture of the BMS, showing MQTT publishers, MQTT broker, Django web interface, and database connections.

VI. RESULTS AND DISCUSSION

Testing confirmed the system's fast response to overcurrent events, with the relay disconnecting within milliseconds when current surpassed 400mA. Data consistently reached the Django web server without delays, validating MQTT for reliable, real-time updates. By aggregating data every 5 seconds and using 1-second averages for trend analysis, network use was optimized without sacrificing detail. The dual-core ESP32 with FreeRTOS maintained a steady 100ms sampling rate, supporting real-time safety monitoring.

FreeRTOS's multitasking, alongside the ESP32's dual-core setup, proved critical. Core 0 managed sensor data collection and relay control, while Core 1 handled MQTT communication. This task separation prevented overloads and ensured smooth, uninterrupted operation, even under high data loads.

Using MQTT for wireless data transmission provided reliable, low-power communication to the Django server, suitable for continuous monitoring. Data transfer was consistent, with real-time updates displayed on battery current, voltage, and relay status. The interface alerted users when current approached unsafe levels, supporting remote monitoring and control.

This project shows that a reliable BMS can be built using accessible components and software. It gives how FreeRTOS, dual-core architecture, and MQTT can work together in a

functional system, creating a basis for battery management projects

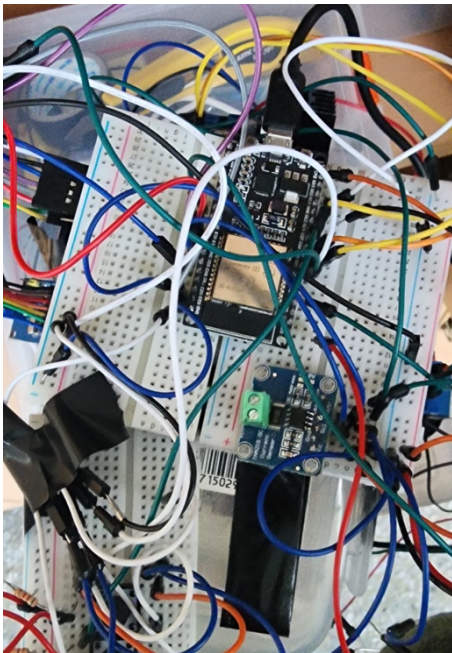


Figure 2. Battery Management System (BMS) device setup with ESP32, INA219 sensor, and relay.

REFERENCES

- [1] W. Puviwatnangkurn, B. Tanboonjit, and N. H. Fuengwarodsakul, "Overcurrent protection scheme of BMS for Li-Ion battery used in electric bicycles," in 2013 10th International Conference on Electrical Engineering/Electronics, Computer, Telecommunications and Information Technology (ECTI-CON), Krabi, Thailand, 2013, pp. 1–5
- [2] R. Y. Kanneboina, G. S. Kumar, R. P. Vidyadhar, V. K. Reddy, and S. Tammineni, "Development of Autonomous Vehicle System using ESP32 Microcontroller," in *2024 Asia Pacific Conference on Innovation in Technology (APCIT)*, Mysore, India, 2024, pp. 1–6.
- [3] B. Neeraja, H. Ralte, K. Jain, S. R. Surlekar, D. A. Arani, and R. C. Mabborang, "Real-Time Life Analysis of the Electric Vehicles using IoT - Computer-Based Smart Systems," in 2023 International Conference on Communication, Security and Artificial Intelligence (ICCSAI), Greater Noida, India, 2023, pp. 238–243
- [4] Woolin Jang, Dongwon Jeong, & Sukhoon Lee (2022). A Method of IoT Object Control based on MQTT for Enhancing Object Mobility. *The Journal of Korean Institute of Information Technology*, 20(3), 107–119
- [5] J. Yu, P. Lou, and X. Wu, "A Dual-Core Real-Time Embedded System for Vision-Based Automated Guided Vehicle," in 2009 IITA International Conference on Control, Automation and Systems Engineering (CASE 2009), Zhangjiajie, China, 2009, pp. 207–211
- [6] Texas Instruments, INA219 Zero-Drift, Bidirectional Current/Power Monitor With I²C Interface, INA219 Datasheet, Rev. November 2017, p. 2.
- [7] https://github.com/SoftBankCorp/ssu_mqtt